

GitHub Actions: Zero to Hero (4 weeks, ~1 hr/day)

Learn by doing. Each stage has: **concepts** -> **example workflow** -> **exercises** -> **checkpoint**.

Setup (15 min, once)

- 1 Create a **new public GitHub repo** (public = free Actions minutes), e.g. gha-practice.
- 2 Copy the app/ folder from this course into the repo root.
- 3 Workflows live in `.github/workflows/` in your repo. For each stage, copy the example from `workflows/` into that folder, commit, push, and watch the **Actions** tab.
- 4 Optional: install `act` (`brew install act`) to run workflows locally, and `actionlint` (`brew install actionlint`) to lint YAML before pushing.

```
mkdir -p .github/workflows
cp workflows/01-hello.yml .github/workflows/
git add . && git commit -m "stage 1" && git push
```

Tip: only keep the stage you're studying active, or the workflows will all fire at once. Disable others from the Actions tab or delete them.

Roadmap

Week	Stage	Topic	File
1	1	Core model: workflow, job, step, runner	01-hello.yml
1	2	Triggers, filters, contexts, expressions, outputs	02-triggers-contexts.yml
2	3	Real CI: needs, matrix, cache, artifacts, concurrency	03-ci-matrix-cache.yml
2	4	Secrets, variables, environments, approvals, OIDC	04-secrets-environments.yml
3	5	Reusable workflows and composite actions	05-*.yml, composite-action-greet.yml
3	6	CI/CD: Docker to GHCR, releases, deploy	06-cicd-deploy.yml
4	7	Advanced: PR automation, dynamic matrix, security, services	07-advanced.yml
4	8	Capstone project	below

Stage 1: The core model

Concepts

- *Workflow* = a YAML file in `.github/workflows/`. *Event* triggers it. It has one or more *jobs*; each job runs on its own fresh *runner* (VM) and has ordered *steps*. A step is either `run:` (shell) or `uses:` (an action).

- Jobs run in parallel by default. Each starts with an empty machine, so you need `actions/checkout`.

Exercises

- 1 Run `01-hello.yml`. Open the run, expand each step, read the logs.
- 2 Click **Run workflow** (`workflow_dispatch`) to trigger it manually.
- 3 Add a second job `goodbye` that runs on `macos-latest`. Note both jobs run in parallel.
- 4 Make a step fail (`run: exit 1`). Observe what happens to later steps.
- 5 Break the YAML indentation on purpose and see how GitHub reports it.

Checkpoint: explain the difference between a workflow, job, step, and runner without notes.

Stage 2: Triggers, contexts, expressions

Concepts

- Events: `push`, `pull_request`, `schedule (cron)`, `workflow_dispatch`, `release`, `issues`, `workflow_run`.
- Filters: `branches`, `tags`, `paths`, `types`.
- `${{ }}` expressions read **contexts**: `github`, `env`, `secrets`, `vars`, `steps`, `needs`, `matrix`, `inputs`, `runner`.
- `if`: conditions, status functions (`always()`, `failure()`), `GITHUB_OUTPUT`, `GITHUB_STEP_SUMMARY`, `GITHUB_ENV`.

Exercises

- 1 Run `02-triggers-contexts.yml` via `push`, then manually with `inputs`.
- 2 Open a PR and confirm the "Only on PRs" step runs.
- 3 Edit only `README.md` and push: it should NOT trigger (`paths-ignore`).
- 4 Add a step that writes `MY_VAR=hi` to `$GITHUB_ENV` and prints it in the *next* step.
- 5 Add a step that only runs when the commit message contains `[deploy]`: `if: contains(github.event.head_commit.message, '[deploy])`.

Checkpoint: you can answer "why is `env.X` empty here?" (scope) and "why did my step output not show?" (`needs id + GITHUB_OUTPUT`).

Stage 3: Real CI

Concepts

- `needs` for job ordering and passing outputs.
- `strategy.matrix` to fan out across OS/versions; `include/exclude`; `fail-fast`.
- `actions/cache` and `setup-*` built-in caching to speed up runs.
- Artifacts to pass files between jobs / keep build outputs.

- concurrency to cancel superseded runs; timeout-minutes; permissions.

Exercises

- 1 Copy `03-ci-matrix-cache.yml`. Confirm the matrix creates 3 test jobs (windows+20 excluded).
- 2 Break a test in `app/app.test.js`. See which jobs fail and which get skipped.
- 3 Run twice and compare cache hit/miss in the logs.
- 4 Download the `dist` artifact from the run summary page.
- 5 Push two commits rapidly and watch the first run get cancelled.
- 6 Add `branches-ignore-style` protection: in repo Settings > Branches, require the test check before merging a PR.

Checkpoint: a PR cannot merge when tests fail.

Stage 4: Secrets and environments

Concepts

- `secrets.*` (encrypted, masked) vs `vars.*` (plain config). Scope: repo, environment, org.
- `GITHUB_TOKEN`: auto-created per run; control with `permissions:.`
- **Environments** add required reviewers, wait timers, branch restrictions, scoped secrets.
- **OIDC**: cloud auth without long-lived keys (`id-token: write`).
- Secrets are not passed to workflows triggered from forked PRs.

Exercises

- 1 Follow the setup notes at the top of `04-secrets-environments.yml`.
- 2 Run it, approve the production deployment from the UI.
- 3 Try echoing the secret. Then try `echo $MY_SECRET | rev` and note masking limits (never print secrets).
- 4 Create environment-specific secrets (staging and production both with `API_URL`) and read them in two jobs.

Checkpoint: explain why OIDC beats stored cloud keys.

Stage 5: Reuse

Concepts

- **Reusable workflow** (`on: workflow_call`, called with `uses:` at the *job* level): share whole pipelines.
- **Composite action** (`action.yml`, `runs.using: composite`): share a sequence of *steps*.
- Later: JavaScript/Docker actions, and publishing to the Marketplace.

Exercises

- 1 Create `.github/actions/greet/action.yml` from `composite-action-greet.yml`.
- 2 Copy both `05-*.yml` files, run the caller.
- 3 Add a new input name to the composite action and thread it through.
- 4 Refactor your Stage 3 `lint + test` into one reusable workflow called from two workflows.

Checkpoint: know when to pick reusable workflow (jobs) vs composite action (steps).

Stage 6: CI/CD

Concepts

- Build and push a container to **GHCR** with `GITHUB_TOKEN` (packages: write).
- Tag-triggered releases (`v1.0.0`), `gh release create`.
- Gate deploys with environments; deploy only from `main`.
- Docker layer caching with `type=gha`.

Exercises

- 1 Add a Dockerfile:

```
FROM node:22-alpine
WORKDIR /app
COPY app/ .
CMD ["node", "-e", "console.log(require('./app').greet('Docker'))"]
```
- 2 Run `06-cicd-deploy.yml`; find the image under your repo's **Packages**.
- 3 `git tag v0.1.0 && git push --tags` and watch the release job.
- 4 Make `deploy-staging` actually deploy somewhere free (GitHub Pages, Fly.io, Render, or a docker run over SSH to a VPS).

Checkpoint: merge to main produces an image; a tag produces a release.

Stage 7: Advanced

Topics in `07-advanced.yml`: PR automation with `gh`, dynamic matrix via `fromJSON`, least-privilege permissions: `{}`, script-injection prevention, service containers.

Exercises

- 1 Open a PR and see the bot comment.
- 2 Change the matrix JSON to 5 services; confirm 5 jobs appear.
- 3 Add a workflow using `on: workflow_run` that fires after CI completes.
- 4 Add **Dependabot** for actions: `.github/dependabot.yml` with `package-ecosystem: github-actions`.
- 5 Add CodeQL scanning (Security tab > Code scanning > set up).
- 6 Install `actionlint` and fix every warning on your workflows.

- 7 Try a **self-hosted runner** on your Mac (Settings > Actions > Runners). Never use these on public repos with untrusted PRs.
-

Stage 8: Capstone

Build a complete pipeline for a small app of your choice:

- [] PR checks: lint, unit tests (matrix), required status check
 - [] Cache and artifacts, concurrency cancel
 - [] Build and push Docker image to GHCR on main
 - [] Auto-deploy to staging; manual approval gate for production
 - [] Tag push creates a release with generated notes
 - [] Reusable workflow plus one composite action in use
 - [] Nightly scheduled job (e.g. dependency audit)
 - [] Dependabot, CodeQL, pinned action SHAs, permissions set everywhere
-

Cheat sheet

```
on: [push]                                # trigger
jobs:
  name:
    runs-on: ubuntu-latest                # runner
    needs: [other]                        # ordering
    if: github.ref == 'refs/heads/main'
    env: { KEY: value }
    permissions: { contents: read }
    steps:
      - uses: actions/checkout@v4         # action
      - run: echo hi                       # shell
        id: x
      - run: echo "k=v" >> "$GITHUB_OUTPUT" # outputs; read: ${ steps.x.outputs.k }
      - run: echo "K=v" >> "$GITHUB_ENV"    # env for later steps
```

Debugging

- Re-run with **Enable debug logging**, or set secret `ACTIONS_STEP_DEBUG=true`.
- `actionlint` catches most syntax and expression mistakes locally.
- `act -j jobname` runs a job locally in Docker (not 100% identical to GitHub).
- Add a `run: env | sort` or `toJSON(github)` step to inspect context.

Resources

- Docs: <https://docs.github.com/actions> (start with *Understanding GitHub Actions and Workflow syntax*)
- Marketplace: <https://github.com/marketplace?type=actions>

- Security hardening:
<https://docs.github.com/actions/security-guides/security-hardening-for-github-actions>
- Starter workflows: <https://github.com/actions/starter-workflows>

Appendix: Example files

Each file below is also in the workflows/ and app/ folders. Copy workflows into .github/workflows/ in your repo.

workflows/01-hello.yml

```
# STAGE 1 - Your first workflow
# Concepts: workflow, event, job, runner, step, `run`, `uses`
name: 01 Hello

on:
  push:
  workflow_dispatch:      # adds a "Run workflow" button in the Actions tab

jobs:
  hello:
    runs-on: ubuntu-latest
    steps:
      - name: Say hello
        run: echo "Hello from GitHub Actions!"

      - name: Show runner info
        run: |
          uname -a
          date
          pwd

      - name: Check out the repo      # without this the runner has NO code
        uses: actions/checkout@v4

      - name: List files
        run: ls -la
```

workflows/02-triggers-contexts.yml

```
# STAGE 2 - Triggers, filters, contexts, expressions, env vars, outputs
name: 02 Triggers and Contexts

on:
  push:
    branches: [main, "feature/**"]
    paths-ignore: ["**/*.md"]
  pull_request:
    branches: [main]
  schedule:
    - cron: "0 6 * * 1"      # Mondays 06:00 UTC
  workflow_dispatch:
  inputs:
    name:
      description: "Who to greet"
      required: true
      default: "world"
    shout:
      type: boolean
      default: false

env:
  WORKFLOW_LEVEL: "visible to all jobs"

jobs:
  inspect:
```

```

runs-on: ubuntu-latest
env:
  JOB_LEVEL: "visible to this job"
steps:
  - name: Dump contexts
    run: |
      echo "event:    ${GITHUB_EVENT_NAME}"
      echo "ref:      ${GITHUB_REF}"
      echo "actor:     ${GITHUB_ACTOR}"
      echo "sha:       ${GITHUB_SHA}"
      echo "runner:    ${RUNNER_OS}"
      echo "inputs:    ${GITHUB_INPUTS}"

  - name: Env vars
    env:
      STEP_LEVEL: "visible to this step"
    run: echo "$WORKFLOW_LEVEL | $JOB_LEVEL | $STEP_LEVEL"

  - name: Only on main
    if: github.ref == 'refs/heads/main'
    run: echo "This is main"

  - name: Only on PRs
    if: github.event_name == 'pull_request'
    run: echo "PR #${GITHUB_EVENT_PULL_REQUEST_NUMBER}"

  - name: Shout (manual runs only)
    if: inputs.shout
    run: echo "${GITHUB_INPUTS}!!!" | tr a-z A-Z

  - name: Set a step output
    id: vers
    run: echo "version=1.2.${GITHUB_RUN_NUMBER}" >> "$GITHUB_OUTPUT"

  - name: Use the output
    run: echo "Version is ${STEPS.VERS.OUTPUTS.VERSION}"

  - name: Job summary (shows on the run page)
    run: |
      echo "### Run summary" >> "$GITHUB_STEP_SUMMARY"
      echo "- version: ${STEPS.VERS.OUTPUTS.VERSION}" >> "$GITHUB_STEP_SUMMARY"

  - name: Status functions
    if: always() # also: success(), failure(), cancelled()
    run: echo "I run even if earlier steps failed"

```

workflows/03-ci-matrix-cache.yml

```

# STAGE 3 - Real CI: multiple jobs, needs, matrix, cache, artifacts, concurrency
name: 03 CI

on:
  push:
    branches: [main]
  pull_request:

# Cancel older runs of the same branch/PR when a new commit arrives
concurrency:
  group: ci-${GITHUB_REF}
  cancel-in-progress: true

permissions:
  contents: read # least privilege for GITHUB_TOKEN

```

```

defaults:
  run:
    working-directory: app

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
      - run: npm run lint

  test:
    needs: lint # runs after lint passes
    runs-on: ${{ matrix.os }}
    timeout-minutes: 10
    strategy:
      fail-fast: false
    matrix:
      os: [ubuntu-latest, windows-latest]
      node: [20, 22]
      exclude:
        - os: windows-latest
          node: 20
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
          # cache: npm # enable once you have package-lock.json
          # cache-dependency-path: app/package-lock.json
      - run: npm test

# Manual caching example (use when setup-* doesn't cache for you)
cache-demo:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/cache@v4
      id: cache
      with:
        path: ~/.cache/demo
        key: demo-${{ runner.os }}-${{ hashFiles('app/package.json') }}
    - if: steps.cache.outputs.cache-hit != 'true'
      run: mkdir -p ~/.cache/demo && date > ~/.cache/demo/built-at.txt
    - run: cat ~/.cache/demo/built-at.txt

build:
  needs: [test]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: npm run build
    - uses: actions/upload-artifact@v4
      with:
        name: dist
        path: app/dist
        retention-days: 7

verify-artifact:
  needs: build

```

```

runs-on: ubuntu-latest
defaults:
  run:
    working-directory: .
steps:
  - uses: actions/download-artifact@v4
    with:
      name: dist
      path: dist
  - run: ls -R dist

```

workflows/04-secrets-environments.yml

```

# STAGE 4 - Secrets, variables, environments, approvals, OIDC
# Setup first (repo > Settings):
#   Secrets and variables > Actions > New secret      : MY_SECRET = anything
#   Secrets and variables > Actions > Variables      : APP_NAME = demo
#   Environments > New environment "production"      : add yourself as Required reviewer
name: 04 Secrets and Environments

```

```

on:
  workflow_dispatch:

```

```

jobs:
  use-secret:
    runs-on: ubuntu-latest
    steps:
      - name: Secrets are masked in logs
        env:
          MY_SECRET: ${ secrets.MY_SECRET }
        run: |
          echo "App name (variable): ${ vars.APP_NAME }"
          echo "Secret length: ${#MY_SECRET}"
          echo "Printing it shows: $MY_SECRET"      # -> ***

```

```

deploy-prod:
  needs: use-secret
  runs-on: ubuntu-latest
  environment:
    name: production      # pauses here until a reviewer approves
    url: https://example.com
  steps:
    - run: echo "Deploying with approval..."

```

```

# OIDC: get short-lived cloud credentials with NO stored secret.
# Example shape for AWS (needs an IAM role trusting GitHub) - read, don't run yet.

```

```

oidc-example:
  if: false
  runs-on: ubuntu-latest
  permissions:
    id-token: write
    contents: read
  steps:
    - uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: arn:aws:iam::123456789012:role/gha-deploy
        aws-region: us-east-1
    - run: aws sts get-caller-identity

```

workflows/05-reusable-called.yml

```

# STAGE 5b - A reusable workflow (trigger: workflow_call)
name: 05 Reusable (called)

```

```

on:
  workflow_call:
    inputs:
      environment:
        type: string
        required: true
    outputs:
      result:
        value: ${ jobs.run.outputs.result }

jobs:
  run:
    runs-on: ubuntu-latest
    outputs:
      result: ${ steps.r.outputs.result }
    steps:
      - uses: actions/checkout@v4
      - uses: ../github/actions/greet          # local composite action
        with:
          who: ${ inputs.environment }
      - id: r
        run: echo "result=deployed-${ inputs.environment }" >> "$GITHUB_OUTPUT"

```

workflows/05-reusable-caller.yml

STAGE 5a - Calls a reusable workflow (needs 05-reusable-called.yml in the same folder)
 name: 05 Reusable Caller

```

on:
  workflow_dispatch:

jobs:
  call-dev:
    uses: ../github/workflows/05-reusable-called.yml
    with:
      environment: dev
    secrets: inherit

  call-staging:
    needs: call-dev
    uses: ../github/workflows/05-reusable-called.yml
    with:
      environment: staging
    secrets: inherit

  show-output:
    needs: call-staging
    runs-on: ubuntu-latest
    steps:
      - run: echo "Reusable returned ${ needs.call-staging.outputs.result }"

```

workflows/06-cicd-deploy.yml

STAGE 6 - Full CI/CD: test -> build+push Docker image to GHCR -> release on tag
 # Requires a Dockerfile at repo root for the `docker` job (a 3-line one is fine).
 name: 06 CI/CD

```

on:
  push:
    branches: [main]
    tags: ["v*.*.*"]
  pull_request:

permissions:

```

```

  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm test
        working-directory: app

  docker:
    needs: test
    if: github.event_name != 'pull_request'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write          # needed to push to ghcr.io
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${GITHUB_ACTOR}
          password: ${GITHUB_TOKEN}
      - id: meta
        uses: docker/metadata-action@v5
        with:
          images: ghcr.io/${GITHUB_REPOSITORY}
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${GITHUB_REF_NAME}
          labels: ${GITHUB_REF_NAME}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  release:
    needs: docker
    if: startsWith(github.ref, 'refs/tags/v')
    runs-on: ubuntu-latest
    permissions:
      contents: write          # create the release
    steps:
      - uses: actions/checkout@v4
      - name: Create GitHub release
        env:
          GH_TOKEN: ${GITHUB_TOKEN}
        run: gh release create "${GITHUB_REF_NAME}" --generate-notes

  deploy-staging:
    needs: docker
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - run: echo "Deploy ghcr.io/${GITHUB_REPOSITORY}:${GITHUB_SHA} to staging"

```

workflows/07-advanced.yml

STAGE 7 - Advanced patterns: PR automation, dynamic matrix, security, hardening

```

name: 07 Advanced

on:
  pull_request:
    types: [opened, synchronize, labeled]
    workflow_dispatch:

permissions: {} # deny everything by default; grant per job

jobs:
  # 1) Auto-label / comment on PRs using the gh CLI
  pr-comment:
    if: github.event_name == 'pull_request'
    runs-on: ubuntu-latest
    permissions:
      pull-requests: write
    steps:
      - env:
          GH_TOKEN: ${ secrets.GITHUB_TOKEN }
        run: |
          gh pr comment "${ github.event.pull_request.number }" \
            --repo "${ github.repository }" \
            --body "Thanks @${ github.actor }! CI is running."

  # 2) Dynamic matrix built from JSON at runtime
  plan:
    runs-on: ubuntu-latest
    outputs:
      matrix: ${ steps.m.outputs.matrix }
    steps:
      - id: m
        run: echo 'matrix={"service":["api","web","worker"]}' >> "$GITHUB_OUTPUT"

  build-services:
    needs: plan
    runs-on: ubuntu-latest
    strategy:
      matrix: ${ fromJSON(needs.plan.outputs.matrix) }
    steps:
      - run: echo "Building ${ matrix.service }"

  # 3) Security: pin third-party actions to a full commit SHA in real projects
  #   uses: owner/action@<40-char-sha> # v1.2.3
  #   And NEVER put untrusted input directly in `run:` (script injection):
  injection-safe:
    runs-on: ubuntu-latest
    steps:
      - name: BAD (do not do this):
        run: echo "${ github.event.pull_request.title }"
        run: echo "see comment above"
      - name: GOOD (pass through env)
        env:
          TITLE: ${ github.event.pull_request.title }
        run: echo "$TITLE"

  # 4) Service container: Postgres alongside your tests
  db-test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: test
        ports: ["5432:5432"]

```

```

    options: >-
      --health-cmd "pg_isready" --health-interval 5s
      --health-timeout 5s --health-retries 5
  steps:
    - run: |
        sudo apt-get install -y postgresql-client >/dev/null
        PGPASSWORD=test psql -h localhost -U postgres -c "select version();"

```

workflows/composite-action-greet.yml

```

# STAGE 5c - A composite action.
# Save this file as: .github/actions/greet/action.yml
name: Greet
description: Prints a greeting (demo composite action)
inputs:
  who:
    description: Who to greet
    required: true
runs:
  using: composite
  steps:
    - shell: bash
      run: echo "Hello, ${inputs.who} - from a composite action"

```

app/app.js

```

// Tiny app used as the practice target for CI/CD workflows.
function add(a, b) {
  return a + b;
}

function greet(name) {
  return `Hello, ${name}!`;
}

module.exports = { add, greet };

```

app/app.test.js

```

const test = require("node:test");
const assert = require("node:assert");
const { add, greet } = require("./app");

test("add", () => assert.strictEqual(add(2, 3), 5));
test("greet", () => assert.strictEqual(greet("Ada"), "Hello, Ada!"));

```

app/package.json

```

{
  "name": "gha-practice-app",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "test": "node --test",
    "lint": "node --check app.js",
    "build": "mkdir -p dist && cp app.js dist/"
  }
}

```